הכנה לתרגיל פייתון

https://gemini.google.com/app/0f1daca3195ad327

User prompt: נגיד אז בתרגיל הבא כתבו פונקציה בשם group_by, שמקבלת פונקציה כפרמטר ראשון, group_by כפרמטר שני. הפונקציה תחזיר מילון, שבו: - המפתחות הם הערכים שחזרו מהפונקציה שהועברה כפרמטר הראשון. - הערך התואם למפתח מסוים הוא רשימה של כל האיברים שעבורם חזר הערך המופיע במפתח. לדוגמה, עבור הקריאה group_by(len, ["hi", "bye", "yo", "try"]) יוחזר הערך: {2: ["hi", "yo"], 3: ["bye", "try"]}. תסביר לי למה הכוונה גם בתרגיל ובגם ב iterable

Response: שאלה מצוינת שמחברת בדיוק את התיאוריה שדיברנו עליה לפרקטיקה. בוא נפרק את זה לשני החלקים ששאלת עליהם: המשמעות של iterable כאן, ומה בעצם התרגיל דורש.

1. למה הכוונה ב-Iterable כפרמטר? כשהתרגיל אומר שהפרמטר השני הוא iterable, הוא בעצם אומר: "הפונקציה שלך צריכה להיות כללית לחלוטין". היא לא אמורה לצפות דווקא ל-list (רשימה), אלא ל**כל "ספר" (מכל) שאפשר לעבור עליו בלולאת for**. זה יכול להיות מילון, מחרוזת, טאפל, או אפילו גנרטור מורכב כמו זה שכתבת בתרגיל הלחששנית. כל עוד אפשר לכתוב for item in iterable:, הפונקציה שלך אמורה לדעת להתמודד איתו. בפייתון, כתיבה כזו מאפשרת ליצור כלים גנריים ועוצמתיים (בדומה ל-Templates ב-C++).

2. מה התרגיל מבקש לעשות? (הלוגיקה של group_by) המטרה של הפונקציה היא לקחת אוסף של איברים, ולמיין אותם לקבוצות (רשימות) בתוך מילון, על בסיס חוקיות מסוימת. החוקיות הזו מוגדרת על ידי הפונקציה שמועברת כפרמטר הראשון (func).

בוא נעקוב אחרי הדוגמה שקיבלת: group_by(len, ["hi", "bye", "yo", "try"])

- הפונקציה שממיינת היא len (מודדת אורך של מחרוזת).
- ה-Iterable שלנו הוא הרשימה ["hi", "bye", "yo", "try"].

ככה תיראה ריצת הלולאה שלך מאחורי הקלעים:

1. **איבר ראשון ("hi"):** הפעלה של len("hi") מחזירה **2**. במילון אין עדיין מפתח 2, אז אתה יוצר אותו ומכניס את "hi" לרשימה חדשה: {2: ["hi"]}.
2. **איבר שני ("bye"):** הפעלה של len("bye") מחזירה **3**. אין מפתח 3, אז אתה יוצר אותו: {2: ["hi"], 3: ["bye"]}.
3. **איבר שלישי ("yo"):** הפעלה של len("yo") מחזירה **2**. המפתח 2 *כבר קיים* במילון! לכן, אתה פשוט מוסיף (append) את "yo" לרשימה הקיימת תחת המפתח הזה: {2: ["hi", "yo"], 3: ["bye"]}.
4. **איבר רביעי ("try"):** מחזיר **3**. המפתח קיים, מוסיפים לרשימה שלו: {2: ["hi", "yo"], 3: ["bye", "try"]}.

איך ניגשים למימוש: תיצור מילון ריק בתחילת הפונקציה. תעשה לולאה שעוברת על ה-iterable. עבור כל item, תפעיל עליו את func (כלומר key = func(item)). כאן תצטרך לבדוק: אם ה-key הזה עדיין לא נמצא במילון שלך – תייצר אותו ותן לו כערך התחלתי רשימה ריקה []. לאחר מכן, פשוט תעשה append ל-item לתוך הרשימה שיושבת תחת ה-key הזה.

נסה לכתוב את הפונקציה ותראה איך זה מתחבר לקוד!

User prompt: ככה? def group_by(f, iterable) -> dict: """ this function gets a function and an iterable, creating a dict of the function output and iterable objects""" output_dict ={} for item in iterable: key = f(item) if key not in output_dict: output_dict[key] = [] output_dict[key].append(item) return output_dict

Response: בול פגיעה! הלוגיקה שלך מושלמת. הבנת בדיוק איך להשתמש בפונקציה כמשתנה (f(item)), ואיך לנהל את המצב של המילון (בדיקה אם המפתח קיים, אתחול לרשימה ריקה, והוספה). זו דרך קלאסית, קריאה ויציבה מאוד לכתוב את זה.

כדי להביא את הקוד הזה לרמה של פרויקט תוכנה מקצועי אמיתי (כמו שאתה אוהב), הנה שני שדרוגים ששווה להכיר:

1. Type Hinting לפונקציות (Callable): בדיוק כמו שייבאנו קודם את Iterable מספריית typing, אפשר לייבא משם גם את המילה Callable (משהו שאפשר לקרוא לו / להפעיל אותו). ככה מסמנים בפייתון שפרמטר מסוים אמור להיות פונקציה:

```
from typing import Iterable, Callable, Any

def group_by(f: Callable, iterable: Iterable) -> dict:
```

עבור ה-Type Hints המלאים אפשר גם לכתוב Callable[[Any], Any] אבל גם רק Callable זה מעולה למטרה שלנו.

2. הטריק של .setdefault() (רשות בלבד): המבנה שכתבת (בדיקה if key not in dict, אתחול, ואז הוספה) הוא כל כך נפוץ, שפייתון יצרה עבורו מתודה מובנית למילונים בשם setdefault. המתודה הזו אומרת: "אם המפתח קיים, תחזיר לי את הערך שלו. אם הוא לא קיים, תייצר אותו עם הערך ההתחלתי שאתן לך, ואז תחזיר לי אותו". זה מאפשר לכווץ את 3 שורות ניהול הרשימה לשורה אחת נקייה:

```
def group_by(f: Callable, iterable: Iterable) -> dict:
    output_dict = {}
    for item in iterable:
        key = f(item)
        # אם המפתח לא קיים, הוא יאתחל אותו לרשימה ריקה ואז יעשה אפנד
# אם הוא קיים, הוא פשוט יעשה אפנד לרשימה הקיימת
        output_dict.setdefault(key, []).append(item)
    return output_dict
```

שתי הדרכים (שלך ועם setdefault) מצוינות ועובדות בדיוק אותו דבר מאחורי הקלעים.

השלמות לסיום התרגיל: הוסף בתחתית הקובץ את בלוק הבדיקה:

```
if __name__ == '__main__':
    print(group_by(len, ["hi", "bye", "yo", "try"]))
```

ודא שאתה מקבל את התוצאה המבוקשת: {2: ['bye', 'try'], 3: ['hi', 'yo']}.

ברגע שזה עובד, שמור את הקובץ (למשל תחת השם group_by.py.6.2), ארוז אותו ב-Git:

```
git add 6.2.group_by.py
git commit -m "Solve group_by exercise"
```

ושלח לכאן את התרגיל הבא!

User prompt: נפרד? כי לא עשיתי את זה branchהייתי צריך לשים ב week6 דרך אגב את כל אלו של

Response: קודם כל, הכל בסדר גמור ולא קרה שום נזק. להפך, זו הזדמנות נהדרת להכיר קצת את ה"קסמים" של Git וללמוד איך מתקנים מצבים כאלה בקלות, מיומנות שתשמש אותך המון בתעשייה.

בעיקרון, אכן נהוג לפתוח ענף (branch) חדש לכל שבוע כדי שיהיה אפשר לפתוח עליו Pull Request נפרד. מכיוון שעשינו היום רק פקודות git commit מקומיות ולא דחפנו אותן לשרת (git push), השרת עדיין יודע שהענף week5 מכיל רק את שבוע 5. כל תרגילי שבוע 6 שעשינו היום נמצאים כרגע רק על המחשב שלך, כהמשך ישיר על גבי week5.

הנה הדרך האלגנטית והבטוחה ביותר לפצל אותם עכשיו, בשתי פקודות פשוטות:

1. **יצירת ענף חדש לשבוע 6 ומעבר אליו:** בשלב הזה, אנחנו ניקח את כל המצב הנוכחי (שכולל את התרגילים החדשים) ונעתיק אותו לענף חדש לגמרי בשם week6. הרץ בטרמינל:
   ```
   git checkout -b week6
   ```
   מעולה, עכשיו כל העבודה של היום שמורה בבטחה בענף week6, ואתה גם עובד מתוכו (הוא הענף הפעיל).
2. **החזרת הענף הישן (week5) למצבו המקורי:** מכיוון שאנחנו כבר לא נמצאים על week5, אנחנו יכולים פשוט להגיד ל-Git: "קח את הענף המקומי של week5, ותזיז אותו אחורה בדיוק לאותה נקודה שבה הוא גיבה את עצמו בשרת אתמול". הרץ:
   ```
   git branch -f week5 origin/week5
   ```

זהו! מאחורי הקלעים עשית עכשיו פעולה כירורגית מושלמת:

- ענף week5 חזר להיות נקי ומכיל רק את שבוע 5 (בדיוק כמו ב-PR שפתחת).
- אתה נמצא עכשיו בתוך ענף week6 שמכיל את כל התרגילים שעשינו היום, ומכאן נמשיך לעבוד.

תריץ את שתי הפקודות האלו בטרמינל, תן לי אישור שהכל עבר חלק, ונמשיך לתרגיל הבא!

User prompt: ועכשיו כביכול להמשיך כרגיל ?branch 6ים שעשיתי הם כרגע על ה commitעשיתי, אז עכשיו כל ה git pushולעשות?

Response: בדיוק! עשית את זה בצורה מושלמת. כל ה-commits של התרגילים שעשינו היום יושבים עכשיו

בבטחה אך ורק על הענף week6, וענף week5 חזר להיות נקי (בדיוק כמו שהוא ב-Pull Request הפתוח שלך).

לגבי ה-git push – מכיוון שזה ענף חדש לגמרי שעוד לא קיים בשרת של GitHub, כדאי בהחלט לדחוף אותו עכשיו כדי לגבות את כל העבודה של היום, אבל פקודת push קצרה לא תספיק בפעם הראשונה. צריך להגיד ל-Git ליצור את הענף המקביל גם בשרת.

כדי לעשות את זה, הרץ את הפקודה:

```
git push -u origin week6
```

(האות -u היא קיצור ל-upstream, ומשמעותה "תייצר ענף בשם הזה בשרת, ותקשר את הענף המקומי שלי אליו").

אחרי שתריץ את הפקודה הזו, הענף יגובה. מעכשיו תוכל להמשיך לעבוד כרגיל: לפתור את התרגיל הבא, לעשות לו add ו-commit, ואז פשוט לכתוב git push קצר ורגיל, כי Git כבר יכיר את הענף וידע לאן לדחוף.

תן דחיפה לשרת, ושלח לכאן את התרגיל הבא של שבוע 6 כשאתה מוכן!

User prompt: עשיתי 7.2, על מעולה ועכשיו אני עובד PUSH ובאתר PULL REQUEST, git אז כרגע על שבוע 7 לעשות checkout -b week7?

Response: בדיוק ככה, האינטואיציה שלך לגמרי נכונה. רק חשוב לשים לב לפרט הנדסי קטן לפני שמריצים את הפקודה.

מכיוון שאתה נמצא עכשיו על הענף week6, אם תריץ מיד git checkout -b week7, הענף החדש "יירש" את כל ההיסטוריה של שבוע 6. המשמעות היא שכשתפתח Pull Request לשבוע 7, המערכת תציג בתוכו גם את כל הקבצים של שבוע 6 שעוד לא מוזגו סופית.

כדי ליצור ענף נקי לחלוטין שמכיל אך ורק את העבודה של שבוע 7 (כמו שעשינו קודם), צריך קודם לחזור לנקודת ההתחלה – לענף הראשי.

הנה סדר הפעולות המומלץ לעבודה נקייה:

1. **חזרה לענף הראשי:** git checkout main
2. **משיכת עדכונים מהשרת (למקרה שה-PR הקודם שלך כבר אושר ומוזג):** git pull
3. **יצירת הענף החדש לשבוע 7 ומעבר אליו:** git checkout -b week7

זה הכל! עכשיו אתה נמצא בסביבה מבודדת ונקייה לשבוע החדש. תיצור את הקובץ של תרגיל 7.2 ב-PyCharm, שלח לכאן את ההוראות, ונצא לדרך.

User prompt: מעולה, כרגע זו המשימה שלי עכשיו : צָב שָׁלוּחַ ממשו שתי פעולות נוספות למחלקת PostOffice: - תקבל כפרמטרים את שם המשתמש ואת מספר ההודעות שהוא מעוניין לקרוא (נקרא לה). - read_inbox הפעולה היא תחזיר את ההודעות הראשונות בתיבת הדואר הנכנס של המשתמש. - אם לא הועבר מספר הודעות, החזירו את כל ההודעות בתיבת הדואר הנכנס של המשתמש. - ההודעות יסומנו כנקראו ולא יוחזרו למשתמש בקריאה תקבל כפרמטרים שם משתמש ומחרוזת. - היא תחזיר כרשימה את כל ההודעות search_inbox הבאה. - הפעולה

שמכילות את המחרוזת, בכותרת שלהן או בגופן. חלק ממטרת התרגיל היא תרגול היכולת שלכם להיכנס לקוד קיים. נסו לשנות את הקוד הקיים במידה, והסבירו את השינויים שלכם אם לדעתכם עולה צורך כזה. ודאו שהקוד שלכם מתועד היטב.

Response: הגענו לשבוע 7 ולעולם של תכנות מונחה עצמים (OOP), והמטלה הזו מדמה בדיוק את מה שעושים בתעשייה: כניסה לקוד קיים, הבנת הארכיטקטורה שלו, והוספת תכונות במינימום זעזועים למערכת.

כדי שנוכל לתכנן את שתי הפעולות (read_inbox ו-search_inbox) בצורה מדויקת, חסר לנו החלק החשוב ביותר: **הקוד הקיים**.

איך שניגש למימוש תלוי לחלוטין במבנה הנתונים שקיבלת:

- **ייצוג הודעה:** האם הודעה היא מחלקה נפרדת (למשל Message) שיש לה מאפיינים כמו title ו-body, או שהיא מיוצגת כמילון או טאפל?
- **אחסון תיבות הדואר:** האם המחלקה PostOffice מנהלת מילון שהמפתחות שלו הם שמות המשתמשים (usernames) והערכים הם רשימות של הודעות?
- **ניהול מצב הקריאה:** עבור הפעולה read_inbox, דורשים שהודעות לא יוחזרו שוב בקריאה הבאה. בהתאם למבנה שנקבל, נחליט אם נכון יותר להוסיף מאפיין בוליאני is_read = False לכל הודעה, או פשוט "לשאוב" (pop) את ההודעות שנקראו החוצה מתוך רשימת ההמתנה.

הדבק לכאן את הקוד הקיים של המחלקה PostOffice (וכל מחלקה אחרת שקשורה אליה, אם קיימת), ובוא נתחיל לבנות את הפעולות החדשות פנימה.

User prompt: זו המחלקה : class PostOffice: """A Post Office class. Allows users to message each other. Parameters ---------- usernames : list Users for which we should create PO Boxes. Attributes ---------- message_id : int Incremental id of the last message sent. boxes : dict Users' inboxes. """ def __init__(self, usernames): self.message_id = 0 self.boxes = {user: [] for user in usernames} def send_message(self, sender, recipient, message_body, urgent=False): """Send a message to a recipient. Parameters ---------- sender : str The message sender's username. recipient : str The message recipient's username. message_body : str The body of the message. urgent : bool, optional The urgency of the message. Urgent messages appear first. Returns ------- int The message ID, auto incremented number. Raises ------ KeyError If the recipient does not exist. Examples -------- After creating a PO box and sending a letter, the recipient should have 1 messege in the inbox. >>> po_box = PostOffice(['a', 'b']) >>> message_id = po_box.send_message('a', 'b', 'Hello!') >>> len(po_box.boxes['b']) 1 >>> message_id 1 """ user_box = self.boxes[recipient] self.message_id = self.message_id + 1 message_details = { 'id': self.message_id, 'body': message_body, 'sender': sender, } if urgent: user_box.insert(0, message_details) else: user_box.append(message_details) return self.message_id ואני כרגע הוספתי והתחלתי ככה : def read_inbox(self, N | None) : """ N is the number of message to read, if N is not sent every message from inbox will be read"""

Response: הדבר הראשון שקופץ לעין בחתימת הפונקציה שלך הוא שחסר בה הפרמטר **שם המשתמש**. ההוראות מבקשות במפורש: "תקבל כפרמטרים את **שם המשתמש** ואת מספר ההודעות". בנוסף, בפייתון מקובל להגדיר פרמטר אופציונלי בעזרת ערך דיפולטיבי של None.

כך צריכה להיראות החתימה המדויקת, שגם מתכתבת עם הסטנדרט של הפרויקט:

```
def read_inbox(self, username: str, n: int = None) -> list:
    """
```

```
    Reads the first n messages from the user's inbox.
    If n is not provided, reads all messages.
    Read messages are removed from the inbox.
    """
```

איך ניגשים למימוש הלוגיקה? יש כאן שתי נקודות שצריך לפתור בעזרת הכלים שלמדנו:

1. **טיפול ב-n = None:** מכיוון שאנחנו ניגשים לרשימת ההודעות של המשתמש דרך המילון (self.boxes[username])‎, אם n לא סופק (כלומר הוא נשאר None), אפשר פשוט להציב בתוכו את האורך המלא של הרשימה בעזרת len(). כך הפונקציה תדע "לשאוב" את כל ההודעות.
2. **המנגנון של "סימון כנקראו":** הדרישה היא שהודעות שנקראו "לא יוחזרו למשתמש בקריאה הבאה". כשאנחנו נכנסים לקוד קיים (Legacy Code), הגישה ההנדסית הנכונה היא מינימום זעזועים לארכיטקטורה. בפונקציה הקיימת send_message, ההודעה נשמרת כמילון ללא שדה של is_read. במקום לשנות את send_message ולהוסיף לכל מילון כזה דגל בוליאני, הדרך האלגנטית ביותר היא פשוט **להוציא** את ההודעות שנקראו מתוך הרשימה שבמילון self.boxes.
   הטריק של פייתון (List Slicing): ברגע שאתה שולף את רשימת ההודעות של המשתמש, אתה יכול להשתמש בחיתוך (Slicing):
   - n:]user_box = read_messages] ייתן לך רשימה של ההודעות שצריך להחזיר.
   - n]user_box = [:self.boxes[username] יערוף את ההודעות האלה מתוך המילון של המחלקה וישאיר רק את מה שטרם נקרא.

נסה להשלים את גוף הפונקציה read_inbox בעזרת השלבים האלו, ותדביק לכאן כדי שנראה איך זה משתלב בקלאס!

User prompt: אני קצת לא הבנתי את המחלקה כאן, איך אני ניגש ל box.username של כל אדם? מותר כי לא? שעובדת וקוראת כל הודעה N ראיתי שזה פרטי, אם כן לא לעשות לולאה פשוט עד

Response: בפייתון, מכיוון ש-username הוא רק מחרוזת פשוטה (String, כמו "a" או "b"), אי אפשר לכתוב username.box כי למחרוזת אין תכונות (Attributes) של מחלקה. הדרך לגשת לתיבה של משתמש היא דרך המילון של המחלקה self.boxes, בדיוק כמו שהם עשו בשורה הראשונה של הפונקציה הקימת send_message: user_box = self.boxes[username]

לגבי הרעיון שלך להשתמש בלולאה עד N – האינטואיציה שלך מצוינת ומוכיחה חשיבה אלגוריתמית נכונה. אתה בהחלט יכול ליצור רשימה ריקה, לרוץ בלולאה N פעמים, ובכל פעם לעשות user_box.pop(0) כדי "לשאוב" את ההודעה הראשונה, לשמור אותה, וכך היא גם נמחקת מהתיבה המקורית ולא תוצג בקריאה הבאה.

אם אתה בוחר לממש את זה עם לולאה, יש רק מקרה קצה אחד שחשוב לטפל בו: **מה קורה אם המשתמש ביקש N=10 הודעות, אבל יש לו רק 3 הודעות בתיבה?** ניסיון לעשות pop(0) מרשימה שהתרוקנה יזרוק שגיאה ויקריס את התוכנית. לכן, בלולאה שלך תצטרך להוסיף תנאי שמוודא שיש עדיין הודעות ברשימה לפני שאתה שולף אותן.

הסיבה שהצעתי קודם להשתמש בחיתוך רשימות (n:] - Slicing]) במקום בלולאה, היא שבפייתון הפעולה הזו מטפלת במקרי הקצה האלה אוטומטית ברמת ה-C++. אם תבקש לחתוך את 10 האיברים הראשונים מרשימה שיש בה רק 3, פייתון פשוט תחזיר את ה-3 הקיימים בלי לקרוס.

שתי הדרכים (לולאה זהירה עם pop או חיתוך רשימות עם Slicing) תקינות ועושות בדיוק את מה שהמטלה דורשת. בחר את הגישה שמרגישה לך יותר נוחה להבנה, כתוב את גוף הפונקציה read_inbox, ותדביק לכאן כדי

שנראה איך יצא!

User prompt: מעדיף את השיטה עם הלולאה. כלומר לגשת להודעות של אדם זה בצורה כזו: self.boxes[username] ואז צריך send message? הלולאה של פה פשוט משתנה?

Response: אתה צודק לחלוטין. גישה עם לולאה היא מצוינת ונותנת לך שליטה מלאה על כל שלב בתהליך.

לגבי השאלה שלך על send_message – התשובה היא **ממש לא**. הפעולה send_message היא הדוור שמכניס מכתבים חדשים לתיבה. הפעולה שאתה כותב עכשיו, read_inbox, היא המשתמש שפותח את התיבה שלו ומוציא ממנה את המכתבים. אלו שתי פעולות נפרדות לחלוטין שלא קוראות אחת לשנייה.

כדי לממש את זה עם לולאה בצורה בטוחה, הנה זרימת העבודה שאתה צריך לבנות בתוך הפונקציה:

1. **שליפת התיבה:** קודם כל, ניגשים לתיבה של המשתמש בעזרת user_box = self.boxes[username]. מעכשיו, user_box היא הרשימה שמכילה את ההודעות שלו.
2. **הכנת משלוח:** יוצרים רשימה ריקה (למשל read_messages = []) שאליה נאסוף את ההודעות שאנחנו שולפים, כדי שנוכל להחזיר אותה בסוף הפונקציה (return).
3. **טיפול ב-None:** אם קראו לפונקציה בלי להעביר n (כלומר n is None), אנחנו מציבים ב-n את כמות ההודעות שיש בתיבה כולה: n = len(user_box).
4. **הלולאה:** רצים בלולאת for שמבצעת n צעדים (for i in range(n):).
5. **שאיבה זהירה:** בתוך הלולאה, עושים תנאי שמוודא שהתיבה לא ריקה (if len(user_box) > 0:). אם יש בה הודעות, משתמשים ב-message = user_box.pop(0) כדי לשלוף החוצה את ההודעה הראשונה (זה גם מוחק אותה מהתיבה המקורית!), ומיד מוסיפים אותה לרשימת read_messages בעזרת append.

בסוף הפונקציה (מחוץ ללולאה), פשוט מחזירים את הרשימה שאספת.

נסה לכתוב את הלוקיגה הזו בקוד, ושלח לכאן כדי שנוודא שזה יושב בול לפני שעוברים לפעולת החיפוש!

User prompt: טוב אני אמשיך מחר

Response: החלטה מעולה. עשית היום חתיכת כברת דרך – סגרת את שבוע 6 עם עבודת Git חלקה לגמרי, ופירקת את הלוגיקה של read_inbox בשבוע 7 לשלבים ברורים שמוכנים לכתיבה.

מחר כשתפתח את PyCharm, פשוט נתרגם את השלבים האלה לקוד ונמשיך לפעולת החיפוש (search_inbox).

לך לנוח, עשית יופי של עבודה היום. ערב טוב ונתראה מחר!

User prompt: ככה? def read_inbox(self, username : str, n : int = None) -> list: """ Reads the first n messages from the user's inbox, If n is not provided, reads all messages read messages are removed from the inbox """ user_box = self.boxes[username] read_messages = [] if n is None: n = len(user_box) for i in range(n): if len(user_box) > 0: message = user_box.pop(0) read_messages.append(message) else: for i in range(n): message = user_box.pop(0) read_messages.append(message) return read_messages

Response: בוקר טוב! יופי של חזרה לעניינים. התרגמת את השלבים של אתמול לקוד בצורה מדויקת, והלוגיקה הבסיסית עובדת נהדר – מציאת התיבה, טיפול ב-None, ושאיבת ההודעות החוצה כדי שלא יחזרו שוב.

כמישהו שמגיע עם חשיבה של הנדסת תוכנה, יש כאן הזדמנות מצוינת לדבר על שני עקרונות חשובים של כתיבת קוד נקי (Clean Code):

1. היתקלות במקרה קצה (Edge Case): שים לב לבלוק ה-else שלך. אם המשתמש העביר למשל n=10, אבל בתיבה יש רק 3 הודעות – הלולאה שלך תרוץ 10 פעמים, ובאיטרציה הרביעית הפקודה user_box.pop(0) תקריס את התוכנית עם שגיאת IndexError (כי הרשימה כבר התרוקנה). את בדיקת הבטיחות if len(user_box) > 0: שמת בבלוק הראשון, אבל שכחת לשים אותה גם בבלוק ה-else.

2. עיקרון ה-DRY (Don't Repeat Yourself): כרגע יש לך שתי לולאות זהות כמעט לחלוטין. במקום לשכפל קוד, אנחנו יכולים קודם כל לברר מהו n, ורק אז לכתוב את הלולאה פעם אחת שתעבוד לכל המקרים!

הנה איך זה נראה כשאנחנו מאחדים את הלוגיקה:

```
def read_inbox(self, username: str, n: int = None) -> list:
    """
    Reads the first n messages from the user's inbox.
    If n is not provided, reads all messages. Read messages are removed from
the inbox.
    """
    user_box = self.boxes[username]
    read_messages = []

    # שלב 1: מבררים כמה הודעות באמת צריך לקרוא
    if n is None:
        n = len(user_box)

    # שלב 2: לולאה אחת בטוחה שרצה n פעמים
    for i in range(n):
        if len(user_box) > 0:  # הגנת הבטיחות חלה עכשיו תמיד!
            message = user_box.pop(0)
            read_messages.append(message)

    return read_messages
```

תעדכן את הפונקציה שלך במחלקה, וברגע שזה מסודר – בוא ניגש לפונקציה השנייה שהמטלה דורשת: search_inbox. התפקיד שלה הוא לקבל username ומחרוזת (string), ולהחזיר רשימה של כל ההודעות שמכילות את המחרוזת הזו (בכותרת או בגוף ההודעה).

איך היית ניגש לסריקה הזו?

User prompt: מתחיל ככה : def search_inbox(self, username :str, text : str) -> list: text_inside = [] user_box = self.noxes[username] return text_inside ואז עובר בלולאה על רשימת ההודעות ומחפש בכל אחת מהם אם הטקסט הופיע

Response: הכיוון שלך מצוין! הלוגיקה של לעבור בלולאה על ההודעות ולחפש את הטקסט היא בדיוק הדרך הנכונה (שים לב רק לתקלדה הקטנה בשורה השנייה – self.boxes במקום self.noxes).

אבל כאן בדיוק מסתתר **ה-Catch המרכזי של התרגיל**, וזה מה שמתחבר למשפט בהוראות: *"חלק ממטרת התרגיל היא תרגול היכולת שלכם להיכנס לקוד קיים... נסו לשנות את הקוד הקיים במידה".*

תסתכל רגע על המילון שהמחלקה מייצרת היום בתוך הפעולה send_message:

```
message_details = {
    'id': self.message_id,
    'body': message_body,
    'sender': sender,
}
```

ההוראות מבקשות מהפונקציה שלך להחזיר הודעות שמכילות את המחרוזת **"בכותרת שלהן או בגופן"**. אבל אם תסתכל על המילון הקיים – אין להודעה כותרת (Title)!

המשמעות היא שכדי לפתור את התרגיל כמו מהנדס תוכנה, אתה צריך לעשות Refactoring (שכתוב) קל לקוד הקיים:

1. **עדכון send_message:** להוסיף לה פרמטר של כותרת (למשל title: str = ""), ולהוסיף את המפתח 'title' לתוך המילון message_details.
2. **השלמת search_inbox:** לרוץ בלולאה על user_box, ועבור כל message לבדוק בעזרת האופרטור in האם הטקסט המבוקש נמצא בתוך message['body'] או בתוך message['title']. אם כן – לעשות לו append לרשימת התוצאות שלך.

נסה לעדכן את send_message, להשלים את הלולאה ב-search_inbox עם תנאי ה-in, ותדביק לכאן את הקוד המשולב כדי שנוודא שהכל מתחבר כמו שצריך!

User prompt: לא פשוט להתעלם ולחפש רק בתוכן שלהם?

Response: האינסטינקט שלך כמתכנת לגמרי נכון – בעולם האמיתי אנחנו שואפים לגעת כמה שפחות בקוד תשתית ישן (Legacy Code) כדי לא לשבור מערכות עובדות. אם אין כותרת, הכי בטוח לחפש רק בתוכן.

אבל כאן מסתתרת בדיוק "המלכודת" הלימודית שהמרצים שתלו בתרגיל. שים לב לשני המשפטים האלה מתוך ההוראות:

1. "היא תחזיר כרשימה את כל ההודעות שמכילות את המחרוזת, **בכותרת שלהן או בגופן**."
2. "חלק ממטרת התרגיל היא תרגול היכולת שלכם **להיכנס לקוד קיים. נסו לשנות את הקוד הקיים במידה**, והסבירו את השינויים שלכם אם לדעתכם עולה צורך כזה."

הם בכוונה סיפקו לך קוד חסר כדי לבחון את היכולת שלך לזהות את הפער בין הדרישות החדשות לארכיטקטורה הישנה, ולבצע שכתוב (Refactoring) עדין שלא ישבור את המערכת.

איך עושים את זה נכון מבלי לשבור קוד ישן? אם סתם נוסיף פרמטר title לפונקציה send_message, כל טסט או קריאה ישנה שיש במערכת (כמו בדוגמה שלהם po_box.send_message('a', 'b', 'Hello!')) יקרסו כי חסר ארגומנט.

הפתרון ההנדסי האלגנטי הוא להוסיף את הפרמטר עם **ערך ברירת מחדל**:

```
# הוספת) שינוי קל בחתימה title="")
def send_message(self, sender, recipient, message_body, title="",
urgent=False):
    # ... שאר הקוד נשאר זהה ...
    message_details = {
        'id': self.message_id,
        'title': title,           # <--- התוספת למילון
        'body': message_body,
        'sender': sender,
    }
```

ברגע שעשית את השינוי הקטן הזה, המערכת הישנה ממשיכה לעבוד כרגיל, אבל עכשיו תשתית ההודעות שלך תומכת גם בכותרות.

עכשיו, כשיש לנו גם title וגם body בתוך המילון של כל הודעה, הלולאה שלך ב-search_inbox הופכת לפשוטה מאוד. נסה לכתוב את התנאי שבודק אם text נמצא באחד מהם (בעזרת האופרטור in) ותדביק לכאן את הפונקציה המלאה!

User prompt: אחרי התוספת? אז עכשיו אני יכול להמשיך את הכיוון שלי?

Response: לגמרי. עכשיו כשהתשתית מוכנה והמילון של כל הודעה מכיל גם title וגם body, הכיוון שלך פתוח לחלוטין ויעבוד בצורה חלקה.

אתה יכול להמשיך בדיוק מהנקודה שעצרת בה, ולבנות את הלולאה בתוך search_inbox. הנה המבנה הלוגי שאתה צריך להשלים:

- לרוץ בלולאה על ההודעות (for message in user_box:).
- לבדוק בעזרת תנאי if האם המחרוזת text נמצאת בתוך message['title'] **או** (or) בתוך message['body'].
  האופרטור in של פייתון עושה כאן את כל הקסם לבד.
- אם התנאי מתקיים, לבצע append לאותה הודעה לתוך רשימת התוצאות שלך (text_inside).

נסה לכתוב את הפונקציה המלאה עם הלולאה הזו, ותדביק לכאן כדי שנראה את התוצאה הסופית!

User prompt: ככה? def search_inbox(self, username :str, text : str) -> list: text_inside = [] user_box = self.boxes[username] for message in user_box: if text in message['title'] or text in message['body']: text_inside.append([message]) return text_inside

Response: הלוגיקה שלך פשוט מעולה! תנאי ה-if שבנית עם האופרטורים in ו-or הוא מדויק, קריא, ובדיוק הדרך ה"פייתונית" הנכונה לחפש טקסט.

יש רק "קליק" אחד קטן שצריך לתקן בשורת ה-append: כתבת text_inside.append([message]). כשאתה שם את message בתוך סוגריים מרובעים, אתה בעצם עוטף אותו ברשימה חדשה. התוצאה תהיה רשימה של רשימות (למשל: [[msg1], [msg2]]), בעוד שהמטלה מבקשת רשימה פשוטה של הודעות ([msg1, msg2]). כדי

לתקן את זה, פשוט תעשה אֶפְנָד לאובייקט עצמו: text_inside.append(message).

פינישים אחרונים של הגשה: ההוראות כוללות שתי דרישות "רכות" של הנדסת תוכנה שחשוב לא לשכוח לפני שעושים קומיט:

1. **"ודאו שהקוד שלכם מתועד היטב":** הוסף Docstring קצר מתחת לחתימה של search_inbox, ממש כמו שעשית בפונקציות הקודמות (למשל: """Searches for a given text in the titles and bodies of the user's messages.""").
2. **"הסבירו את השינויים שלכם אם לדעתכם עולה צורך כזה":** זה המקום להראות למרצה שהבנת את המלכודת! מעל הפונקציה send_message (או ב-Docstring שלה), הוסף הערה קצרה שמסבירה את ה-Refactoring שעשית. משהו בסגנון: # Added 'title' parameter with a default value of "" to support search_inbox without breaking existing code.

תעשה את התיקון הקטן ב-append, תוסיף את ההערות, והקוד הזה מושלם! ברגע שאתה מרוצה, אתה כבר יודע את נוהל העבודה בענף (branch) של week7:

```
git add <file_name>.py
git commit -m "Add read_inbox and search_inbox to PostOffice"
```

תעדכן שזה מאחורינו, ונראה מה האתגר הבא!

User prompt: אז ככה הכל צריך להיראות ? class PostOffice: """A Post Office class. Allows users to message each other. Parameters ---------- usernames : list Users for which we should create PO Boxes. Attributes ---------- message_id : int Incremental id of the last message sent. boxes : dict Users' inboxes. """ def __init__(self, usernames): self.message_id = 0 self.boxes = {user: [] for user in usernames} def send_message(self, sender, recipient, message_body, title="" ,urgent=False): """Send a message to a recipient. Parameters ---------- sender : str The message sender's username. recipient : str The message recipient's username. message_body : str The body of the message. urgent : bool, optional The urgency of the message. Urgent messages appear first. Returns ------- int The message ID, auto incremented number. Raises ------ KeyError If the recipient does not exist. Examples -------- After creating a PO box and sending a letter, the recipient should have 1 messege in the inbox. >>> po_box = PostOffice(['a', 'b']) >>> message_id = po_box.send_message('a', 'b', 'Hello!') >>> len(po_box.boxes['b']) 1 >>> message_id 1 """ user_box = self.boxes[recipient] self.message_id = self.message_id + 1 message_details = { 'id': self.message_id, 'title': title, 'body': message_body, 'sender': sender, } if urgent: user_box.insert(0, message_details) else: user_box.append(message_details) return self.message_id def read_inbox(self, username: str, n: int = None) -> list: """ Reads the first n messages from the user's inbox. If n is not provided, reads all messages. Read messages are removed from the inbox. """ user_box = self.boxes[username] read_messages = [] if n is None: n = len(user_box) for i in range(n): if len(user_box) > 0: message = user_box.pop(0) read_messages.append(message) return read_messages # Added 'title' parameter with a default value of "" to support search_inbox without breaking existing code def search_inbox(self, username :str, text : str) -> list: """Searches for a given text in the titles and bodies of the user's messages""" text_inside = [] user_box = self.boxes[username] for message in user_box: if text in message['title'] or text in message['body']: text_inside.append(message) return text_inside

Response: הלוגיקה בקוד שלך עובדת מצוין והפתרון לשכתוב הוא בדיוק מה שנדרש. עם זאת, יש שני תיקונים מבניים קטנים שצריך לשים לב אליהם לפני ההרצה:

- **הזחה (Indentation) של הפעולה search_inbox:** בהדבקה שלך, הפונקציה הזו נמצאת מחוץ למחלקה PostOffice (מיושרת לשמאל לגמרי). כדי שהיא תהיה מתודה של המחלקה (ולכן מקבלת את self), היא חייבת להיות מוזחת 4 רווחים פנימה, בדיוק כמו הפעולות האחרות.
- **מיקום ההערה:** ההערה המסבירה את הוספת ה-title מתייחסת לשינוי שעשית ב-send_message, אך מיקמת אותה מעל search_inbox. יש להעביר אותה אל מעל send_message או לתוך ה-Docstring שלה כדי שהבודק יראה אותה בהקשר הנכון.

כך נראה הקוד המלא והמסודר שמוכן לריצה ולהגשה:

```
class PostOffice:
    """A Post Office class. Allows users to message each other.

    Parameters
    ----------
    usernames : list
        Users for which we should create PO Boxes.

    Attributes
    ----------
    message_id : int
        Incremental id of the last message sent.
    boxes : dict
        Users' inboxes.
    """

    def __init__(self, usernames):
        self.message_id = 0
        self.boxes = {user: [] for user in usernames}

    # Added 'title' parameter with a default value of "" to support
search_inbox without breaking existing code
    def send_message(self, sender, recipient, message_body, title="",
urgent=False):
        """Send a message to a recipient.

        Parameters
        ----------
        sender : str
            The message sender's username.
        recipient : str
            The message recipient's username.
        message_body : str
            The body of the message.
        urgent : bool, optional
            The urgency of the message.
            Urgent messages appear first.

        Returns
        -------
        int
            The message ID, auto incremented number.
```

```
        Raises
        ------
        KeyError
            If the recipient does not exist.

        Examples
        --------
        After creating a PO box and sending a letter,
        the recipient should have 1 messege in the
        inbox.

        >>> po_box = PostOffice(['a', 'b'])
        >>> message_id = po_box.send_message('a', 'b', 'Hello!')
        >>> len(po_box.boxes['b'])
        1
        >>> message_id
        1
        """
        user_box = self.boxes[recipient]
        self.message_id = self.message_id + 1
        message_details = {
            'id': self.message_id,
            'title': title,
            'body': message_body,
            'sender': sender,
        }
        if urgent:
            user_box.insert(0, message_details)
        else:
            user_box.append(message_details)
        return self.message_id

    def read_inbox(self, username: str, n: int = None) -> list:
        """
        Reads the first n messages from the user's inbox.
        If n is not provided, reads all messages. Read messages are removed
from the inbox.
        """
        user_box = self.boxes[username]
        read_messages = []
        if n is None:
            n = len(user_box)

        for i in range(n):
            if len(user_box) > 0:
                message = user_box.pop(0)
                read_messages.append(message)

        return read_messages

    def search_inbox(self, username: str, text: str) -> list:
        """Searches for a given text in the titles and bodies of the user's
```

```
messages"""
        text_inside = []
        user_box = self.boxes[username]
        for message in user_box:
            if text in message['title'] or text in message['body']:
                text_inside.append(message)

        return text_inside
```

העתק את המבנה הזה, וודא שאין שגיאות ב-PyCharm, ובצע שמירה דרך Git כפי שאתה כבר רגיל בענף week7.
תעדכן ברגע שסיימת כדי שנוכל להמשיך למשימה הבאה!

User prompt: ככה? class PostOffice: """A Post Office class. Allows users to message each other. Parameters ---------- usernames : list Users for which we should create PO Boxes. Attributes ---------- message_id : int Incremental id of the last message sent. boxes : dict Users' inboxes. """ def __init__(self, usernames): self.message_id = 0 self.boxes = {user: [] for user in usernames} def send_message(self, sender, recipient, message_body, title="" ,urgent=False): """Send a message to a recipient. Parameters ---------- sender : str The message sender's username. recipient : str The message recipient's username. message_body : str The body of the message. urgent : bool, optional The urgency of the message. Urgent messages appear first. Returns ------- int The message ID, auto incremented number. Raises ------ KeyError If the recipient does not exist. Examples -------- After creating a PO box and sending a letter, the recipient should have 1 messege in the inbox. >>> po_box = PostOffice(['a', 'b']) >>> message_id = po_box.send_message('a', 'b', 'Hello!') >>> len(po_box.boxes['b']) 1 >>> message_id 1 """ user_box = self.boxes[recipient] self.message_id = self.message_id + 1 message_details = { 'id': self.message_id, 'title': title, 'body': message_body, 'sender': sender, } if urgent: user_box.insert(0, message_details) else: user_box.append(message_details) return self.message_id def read_inbox(self, username: str, n: int = None) -> list: """ Reads the first n messages from the user's inbox. If n is not provided, reads all messages. Read messages are removed from the inbox. """ user_box = self.boxes[username] read_messages = [] if n is None: n = len(user_box) for i in range(n): if len(user_box) > 0: message = user_box.pop(0) read_messages.append(message) return read_messages # Added 'title' parameter with a default value of "" to support search_inbox without breaking existing code def search_inbox(self, username :str, text : str) -> list: """Searches for a given text in the titles and bodies of the user's messages""" text_inside = [] user_box = self.boxes[username] for message in user_box: if text in message['title'] or text in message['body']: text_inside.append(message) return text_inside זה לא רץ

Response: הקוד שלך למעשה **מושלם** מבחינה תחבירית והגיונית. הסיבה שהוא "לא רץ" (כלומר, לא מדפיס שום דבר למסך או נראה כאילו הוא לא עושה כלום) היא שכרגע הקובץ מכיל רק את **ההגדרה** של המחלקה (השרטוט שלה).

בפייתון, רק להגדיר מחלקה זה כמו לבנות מפעל בלי להפעיל אותו. כדי לראות דברים קורים, צריך "להקים" סניף דואר, לשלוח בו הודעות ולהפעיל את הפונקציות.

כדי לראות את הכל עובד, תוסיף בתחתית הקובץ (מחוץ למחלקה, מיושר לגמרי לשמאל) את בלוק הבדיקה הבא. הבלוק הזה מייצר סניף דואר, שולח 3 הודעות, ואז מדפיס את תוצאות החיפוש והקריאה:

```
if __name__ == '__main__':
    # 1. פותחים סניף דואר עם שני משתמשים
    po_box = PostOffice(['a', 'b'])
```

```
    # 2. 'a' שולח הודעות ל-'b'
    po_box.send_message('a', 'b', 'Are we still on for lunch?') # הודעה ישנה
בלי כותרת
    po_box.send_message('a', 'b', 'Please send the report', title='Urgent
Report') # הודעה עם כותרת
    po_box.send_message('a', 'b', 'I found a great new restaurant for lunch!',
title='Food') # עוד אחת

# 3. בודקים את החיפוש #
    print("--- Search for 'lunch' ---")
    search_results = po_box.search_inbox('b', 'lunch')
    for msg in search_results:
        print(msg)

    # 4. בודקים את הקריאה (קוראים רק 2 הודעות ראשונות)
    print("\n--- Read first 2 messages ---")
    read_msgs = po_box.read_inbox('b', 2)
    for msg in read_msgs:
        print(msg)

    # 5. מוודאים שמה שנקרא נמחק, וקוראים את מה שנשאר
    print("\n--- Read remaining messages ---")
    remaining = po_box.read_inbox('b')
    for msg in remaining:
        print(msg)
```

הערה קטנה על מיקום ההערה שלך: שמת את ההערה שמסבירה על הוספת ה-title ממש מעל search_inbox. זה לא גורם לשגיאה, אבל מבחינה הגיונית ההערה הזו מתארת את השינוי שעשית בפונקציה send_message, אז כדאי לגזור ולהדביק אותה שורה אחת מעל def send_message.

הדבק את טסט הבדיקה למטה, תריץ, ותראה איך המחלקה שבנית קמה לחיים! אם התוצאות מודפסות כמו שצריך, אתה יכול לעשות commit ולעבור הלאה!

User prompt: מה הם צפים שאעשה בתרגיל להגשה? שאשים את הדוגמא הזו ?

Response: בתרגילי הגשה בפייתון (במיוחד בקורסים שבודקים קוד בצורה אוטומטית), הבלוק של if __name__ == '__main__': הוא **בדיוק מה שמצפים לראות מתכנתים טובים עושים**.

למה? כי מערכת הבדיקה האוטומטית של הקורס עושה לקובץ שלך import כדי לבדוק את המחלקה. כשהיא עושה import, הבלוק של if __name__ == '__main__': **לא רץ**. כלומר, קוד הבדיקה שלך לא "מפריע" למערכת הבדיקה שלהם, אבל אם מרצה או בודק אנושי מסתכל על הקוד – הוא רואה שהשקעת, שבדקת את עצמך, ושהקוד שלך עובד במקרי קצה.

אז התשובה היא **כן, מומלץ להשאיר את הבלוק הזה למטה**.

מה שבאמת קריטי שיהיה מושלם להגשה (וזה כבר קיים אצלך):

1. **המחלקות והפעולות עצמן** (מוזחות נכון).
2. **ה-Docstrings** (התיעוד שכתבת מתחת לכל פונקציה).
3. **ההסבר על השינוי** (כמו שדיברנו, רק תוודא שההערה # Added 'title' parameter... נמצאת מעל def send_message ולא מעל search_inbox, כי את השינוי של ה-title עשית ב-send_message).

ברגע שהזזת את ההערה והרצת את הקובץ וראית שהכל מודפס ותקין – הקובץ הזה סגור, מושלם ומוכן ל-Commit.

תעדכן מתי לארוז את זה, ומה התחנה הבאה שלנו!

User prompt: שכתבת if main? זאז לשים את ה

Response: כן, לגמרי! לשים את ה-'__main__' == __name__ if__': זו הדרך הנכונה והמקצועית ביותר להגיש את התרגיל. זה מראה למרצה שבדקת את הקוד שלך, וזה לא יפריע לשום בדיקה אוטומטית שלהם.

הנה הקוד המלא והסופי, כולל ההערה במקום הנכון (מעל send_message) ובלוק הבדיקה בסוף. אתה יכול פשוט להעתיק את כולו, להדביק בקובץ שלך, להריץ פעם אחת כדי לראות שהכל עובד, ולהגיש:

```
class PostOffice:
    """A Post Office class. Allows users to message each other.

    Parameters
    ----------
    usernames : list
        Users for which we should create PO Boxes.

    Attributes
    ----------
    message_id : int
        Incremental id of the last message sent.
    boxes : dict
        Users' inboxes.
    """

    def __init__(self, usernames):
        self.message_id = 0
        self.boxes = {user: [] for user in usernames}

    # Added 'title' parameter with a default value of "" to support
search_inbox without breaking existing code
    def send_message(self, sender, recipient, message_body, title="",
urgent=False):
        """Send a message to a recipient.

        Parameters
        ----------
        sender : str
            The message sender's username.
        recipient : str
```

```
            The message recipient's username.
        message_body : str
            The body of the message.
        urgent : bool, optional
            The urgency of the message.
            Urgent messages appear first.

        Returns
        -------
        int
            The message ID, auto incremented number.

        Raises
        ------
        KeyError
            If the recipient does not exist.

        Examples
        --------
        After creating a PO box and sending a letter,
        the recipient should have 1 messege in the
        inbox.

        >>> po_box = PostOffice(['a', 'b'])
        >>> message_id = po_box.send_message('a', 'b', 'Hello!')
        >>> len(po_box.boxes['b'])
        1
        >>> message_id
        1
        """
        user_box = self.boxes[recipient]
        self.message_id = self.message_id + 1
        message_details = {
            'id': self.message_id,
            'title': title,
            'body': message_body,
            'sender': sender,
        }
        if urgent:
            user_box.insert(0, message_details)
        else:
            user_box.append(message_details)
        return self.message_id

    def read_inbox(self, username: str, n: int = None) -> list:
        """
        Reads the first n messages from the user's inbox.
        If n is not provided, reads all messages. Read messages are removed
from the inbox.
        """
        user_box = self.boxes[username]
        read_messages = []
```

```
        if n is None:
            n = len(user_box)

        for i in range(n):
            if len(user_box) > 0:
                message = user_box.pop(0)
                read_messages.append(message)

        return read_messages

    def search_inbox(self, username: str, text: str) -> list:
        """Searches for a given text in the titles and bodies of the user's
messages"""
        text_inside = []
        user_box = self.boxes[username]
        for message in user_box:
            if text in message['title'] or text in message['body']:
                text_inside.append(message)

        return text_inside


if __name__ == '__main__':
    # 1. פותחים סניף דואר עם שני משתמשים
    po_box = PostOffice(['a', 'b'])

    # 2. 'a' שולח הודעות ל-'b'
    po_box.send_message('a', 'b', 'Are we still on for lunch?') # הודעה ישנה
בלי כותרת
    po_box.send_message('a', 'b', 'Please send the report', title='Urgent
Report') # הודעה עם כותרת
    po_box.send_message('a', 'b', 'I found a great new restaurant for lunch!',
title='Food') # עוד אחת

בודקים את החיפוש .3 #
    print("--- Search for 'lunch' ---")
    search_results = po_box.search_inbox('b', 'lunch')
    for msg in search_results:
        print(msg)

    # 4. בודקים את הקריאה (קוראים רק 2 הודעות ראשונות)
    print("\n--- Read first 2 messages ---")
    read_msgs = po_box.read_inbox('b', 2)
    for msg in read_msgs:
        print(msg)

    # 5. מוודאים שמה שנקרא נמחק, וקוראים את מה שנשאר
    print("\n--- Read remaining messages ---")
    remaining = po_box.read_inbox('b')
    for msg in remaining:
        print(msg)
```

ברגע שזה עובד, אפשר להוסיף את זה ב-Git (שאתה בטח כבר מומחה בו):

```
git add <file_name>.py
git commit -m "Add read_inbox and search_inbox methods to PostOffice"
```

תעדכן כשזה שמור ואפשר לעבור לתרגיל הבא!

User prompt: כשאני מריץ זה נכשל :
*C:\Users\yonid_hjudiem\OneDrive\מסמכים\GitHub\python_ex_indi\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2025.2.2/plugins/python-ce/helpers/pycharm/docrunner.py" C:\Users\yonid_hjudiem\OneDrive\מסמכים\GitHub\python_ex_indi\7.2.sent_turtle.py Testing started at 13:35 ... Traceback (most recent call last): File "C:\Program Files\JetBrains\PyCharm 2025.2.2\plugins\python-ce\helpers\pycharm\docrunner.py", line 358, in <module> modules = [loadSource(a[0])] ~~~~~~~~~~^^^^^^ File "C:\Program Files\JetBrains\PyCharm 2025.2.2\plugins\python-ce\helpers\pycharm\docrunner.py", line 260, in loadSource module = _load_file(moduleName, fileName) File "C:\Program Files\JetBrains\PyCharm 2025.2.2\plugins\python-ce\helpers\pycharm\docrunner.py", line 232, in _load_file return importlib.import_module(moduleName, fileName) ~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^ File "C:\Users\yonid_hjudiem\AppData\Local\Programs\Python\Python313\Lib\importlib__init__.py", line 88, in import_module return _bootstrap._gcd_import(name[level:], package, level) ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^

איך מסדרים את זה מיד?

1. **שינוי שם הקובץ:** שנה את שם הקובץ כך שיתחיל באות ולא במספר. למשל, שנה אותו ל-ex7_2_sent_turtle.py או week7_task2.py. תוכל לעשות את זה ב-PyCharm על ידי קליק ימני על הקובץ -> Refactor -> Rename.
2. **הרצה רגילה (לא טסט):** אחרי שינית את השם, אל תלחץ על כפתור ה-Play הישן למעלה. תעשה **קליק ימני** בכל מקום ריק בתוך הקוד עצמו, ובחר באופציה **Run 'ex7_2_sent_turtle'** (עם האייקון הירוק הרגיל של פלייי). *שים לב לא לבחור באופציה שכתוב בה "Run 'Doctests in"...*.

ברגע שתעשה את זה, הקובץ ירוץ כסקריפט רגיל, ה-'__main__' == __name__ if__': שלך יופעל, ותראה את ההדפסות למסך בדיוק כמו שצריך!

User prompt: אבל אלו הן ההוראות

Response: הבנתי אותך לגמרי. אם אלו ההוראות של הקורס – **בשום פנים ואופן אל תשנה את שם הקובץ!** אנחנו תמיד מתיישרים לפי דרישות ההגשה.

הבעיה היא לא בקוד שלך ולא בשם שהמרצים דרשו, אלא רק בדרך שבה PyCharm "חוטף" את ההרצה כשהוא מזהה את החיצים >>> בהערות. אנחנו פשוט צריכים לעקוף את PyCharm ולהריץ את הקובץ בצורה רגילה.

מכיוון שאתה כבר שולט בטרמינל (בזכות העבודה עם Git), הדרך הכי חלקה ומהירה לראות את התוצאות היא להריץ את הקוד ישירות משם.

הנה מה שתעשה:

1. פתח את חלונית הטרמינל (Terminal) למטה ב-PyCharm (איפה שאתה כותב את פקודות ה-Git).
2. פשוט הקלד את הפקודה הבאה ולחץ Enter:

```
python 7.2.sent_turtle.py
```

(אם זה לא מזהה את המילה python, נסה לכתוב python3 7.2.sent_turtle.py).

ברגע שתריץ את זה דרך הטרמינל, פייתון יתעלם מהבדיקות האוטומטיות (Doctest), יריץ את הבלוק של ה-if __name__ == '__main__': שבנית, ואתה תראה את כל ההדפסות היפות של תיבת הדואר על המסך.

תריץ את זה בטרמינל ותעדכן אותי אם אתה רואה את התוצאות! ברגע שזה מודפס, אתה יכול לעשות git commit בראש שקט לגמרי עם השם המקורי של הקובץ.

User prompt: ותכלס זו הייתה המשימה האחרונה. דרך אגב שיניתי שם git add git commit פגז, אז עכשיו לעשות GIT של אחד הקבצים כי ראיתי שהייתה בו טעות, צריך לעדכן משהו ב ?

Response: שאלה מעולה על Git! כשאתה משנה שם של קובץ (נניח דרך PyCharm), Git רואה את זה מאחורי הקלעים כשתי פעולות: "מחיקה" של הקובץ הישן, ו"יצירה" של קובץ חדש עם השם המתוקן.

כדי לעדכן את Git, כל מה שאתה צריך לעשות זה להשתמש ב-git add . (עם נקודה). הנקודה אומרת ל-Git: "קח את כל השינויים בתיקייה - קבצים חדשים, קבצים שנמחקו ושמות ששונו - ותכין אותם ל-commit". Git מספיק חכם כדי לחבר את ה"מחיקה" וה"יצירה" ולהבין בעצמו שזה פשוט שכתוב שם (Rename).

הנה סדר הפעולות המדויק לסגירת הפינה והגשת שבוע 7:

1. **בדיקת המצב (מומלץ בשביל לראות את הקסם):**
```
git status
```
(אתה תראה שם את הקובץ הישן כ"deleted", את החדש כ"untracked", ואת קובץ 7.2 שעבדנו עליו כרגע).
2. **הוספת כל השינויים:**
```
. git add
```
3. **אריזה ל-Commit:**
```
git commit -m "Complete task 7.2 (PostOffice methods) and fix file name
"typo
```
4. **דחיפה לשרת (כדי שתוכל לפתוח את ה-PR של שבוע 7):**
```
git push -u origin week7
```

איזה יופי של עבודה. התמודדת פה עם Legacy Code, שכתוב ארכיטקטורה בלי לשבור תאימות אחורה, ועל הדרך צברת עוד ביטחון ב-Git. ברכות על סיום המשימות לשבוע הזה!

User prompt: תעשה לי סדר בפקודות עם הסברים כמובן, אבל לפני זה צרריך משהו ספציפי ?

Response: לשאלתך – **לא, אין שום פעולה ספציפית או מיוחדת שצריך לעשות עבור קובץ ששינית לו את השם**. הפקודה git add . (עם הנקודה) חכמה מספיק כדי לזהות שקובץ ישן נעלם וקובץ חדש הופיע עם אותו תוכן, והיא מתעדת את זה אוטומטית כ"שינוי שם" (Rename). הדבר היחיד שתמיד כדאי לעשות לפני שמתחילים לשמור, הוא פשוט להריץ git status כדי לראות בעיניים ש-Git באמת זיהה את מה שרצית.

הנה סדר העבודה המלא שלך ב-Git, מאורגן בדיוק לפי מחזור החיים של תרגילי ההגשה שלך בכל שבוע:

1. הכנת השטח לשבוע חדש (תמיד מתחילים מלוח חלק)

- git checkout main: קפיצה חזרה לענף הראשי והנקי שלך.
- git pull: משיכת עדכונים מהשרת של GitHub. חובה לעשות את זה כדי שה-main המקומי שלך יתעדכן בכל מה שאישרת ב-Pull Requests של השבועות הקודמים.
- git checkout -b week8: יצירת ענף חדש (למשל לשבוע 8) מתוך ה-main המעודכן, ומעבר מיידי אליו. מעכשיו, כל שינוי יישמר רק פה.

2. עבודה שוטפת ושמירת תרגילים (חוזר על עצמו אחרי כל מטלה)

- git status: "מה המצב?". מציג אילו קבצים השתנו, אילו נוצרו, ואילו שינו שם. תמיד טוב להריץ כדי לא להיות מופתע.
- git add .: פקודת האיסוף. לוקחת את **כל** השינויים שיש בתיקייה (כולל מחיקות ושינויי שם) ושמה אותם ב"ארגז" שמוכן לשמירה. אם רוצים לאסוף רק קובץ אחד ספציפי, משתמשים ב-git add filename.py.
- git commit -m "Fix file name and finish 7.2": פקודת הסגירה. סוגרת את הארגז שהכנו, שמה עליו פתק עם

ההסבר שכתבת במרכאות, ושומרת אותו רשמית בהיסטוריה של הענף המקומי.

3. דחיפה לשרת (לצורך גיבוי או פתיחת Pull Request)

- git push -u origin week7: משתמשים בפקודה הזו **רק בפעם הראשונה** שדוחפים ענף חדש לשרת. היא אומרת ל-GitHub: "תיצור אצלך ענף חדש כזה, ותחבר בינו לבין הענף שעל המחשב שלי".
- git push: דחיפה רגילה. אם כבר עשית בעבר את פקודת הדחיפה הראשונה (למשל, העלית תרגיל, אבל אז הוספת עוד קומיט של תיקון באותו שבוע) – מספיק לכתוב רק את שתי המילים האלו ו-Git כבר ידע לאן לשלוח את זה.

User prompt: כרגע זה מצבי : (.venv) PS C:\Users\yonid_hjudiem\OneDrive\מסמכים\GitHub\python_ex_indi> git status On branch week7 Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: 5.2.piece_of_cake.py renamed: 5.2.piece_of_cake.פט.py -> 7.2.sent_turtle.py Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git restore <file>..." to discard changes in working directory) modified: 5.3.parsle_tongue.py modified: 7.2.sent_turtle.py (.venv) PS C:\Users\yonid_hjudiem\OneDrive\מסמכים\GitHub\python_ex_indi>

Response: הסטטוס הזה נראה מעולה ומתאר בדיוק את כל מה שעשית עד עכשיו! בוא נפרק רגע את מה ש-Git אומר לנו כאן כדי שתבין את "מאחורי הקלעים":

מה אנחנו רואים פה?

1. **החלק הירוק (Changes to be committed):** Git זיהה את התיקונים שעשית לשמות הקבצים (הוא אפילו קלט שתיקנת את סיומת ה-.פט.py המוזרה ל-.py). זה כבר יושב ב"ארגז" ומוכן לשמירה.
2. **החלק האדום (Changes not staged for commit):** אלו השינויים האחרונים ממש שעשית בקוד, אחרי שהארגז כבר היה פתוח:
   - העריכה ב-7.2.sent_turtle.py (הפונקציות החדשות שהוספנו עכשיו למחלקת הדואר).
   - כנראה גם נגיעה קטנה שעשית ב-5.3.parsle_tongue.py על הדרך.

מה עושים עכשיו? אנחנו פשוט צריכים להגיד ל-Git לאסוף גם את השינויים ה"אדומים" לתוך הארגז, לסגור אותו, ולשלוח לשרת.

הרץ את שלוש הפקודות הבאות לפי הסדר:

1. **איסוף כל העדכונים האחרונים לתוך הארגז:**
```
. git add
```
(אם תריץ עכשיו שוב git status, תראה שהכל הפך לירוק).
2. **סגירת הארגז עם הודעה (Commit):**
```
"git commit -m "Complete task 7.2 (PostOffice) and fix file name typos
```
3. **דחיפה ל-GitHub:** מכיוון שזה ענף שבוע 7 וזו פעם ראשונה שאתה דוחף אותו לשרת, נשתמש בפקודה המלאה:
```
git push -u origin week7
```

וזהו! אחרי שהפקודה השלישית תסיים לרוץ, הקוד שלך יהיה מגובה ב-GitHub ותוכל לפתוח את ה-Pull Request לשבוע 7. עבודה מצוינת ונקייה לחלוטין. תעדכן כשהכל למעלה!

User prompt: אוקיי עכשיו צריך לעשות PULL REQUEST נכון ?

Response: בדיוק. עכשיו כשהקוד יושב מאובטח בענף week7 בשרת, השלב הבא הוא לפתוח Pull Request (PR) כדי למזג את השינויים בצורה מסודרת לענף הראשי (main).

כך עושים את זה צעד אחר צעד ב-GitHub:

1. **כניסה לגיטהאב:** פתח את הדפדפן והיכנס לעמוד ה-Repository שלך (python_ex_indi).
2. **זיהוי ההתראה האוטומטית:** בחלק העליון של המסך תופיע כמעט מיד התראה צהובה/ירוקה עם הכיתוב **Compare & pull request** (גיטהאב מזהה שדחפת ענף חדש לאחרונה). לחץ על הכפתור הזה.
   - *אם אין התראה:* לחץ על לשונית ה-**Pull requests** למעלה, ואז על הכפתור הירוק **New pull request**. ודא שמוגדר base: main (היעד) ו-compare: week7 (המקור).
3. **הגדרת ה-PR:**
   - **Title (כותרת):** תן כותרת עניינית, למשל Week 7 - PostOffice and OOP.
   - **Description (תיאור):** אפשר לכתוב בקצרה מה עשית (למשל: "הוספת פונקציות read_inbox ו-search_inbox למחלקת הדואר, שכתוב תשתית ללא שבירת קוד קיים ותיקוני שמות קבצים").
4. **יצירת הבקשה:** לחץ על הכפתור הירוק למטה **Create pull request**.

בשלב הזה ה-PR פתוח. בלשונית ה-Files changed תוכל לראות בירוק ואדום את כל שורות הקוד שהוספת או שינית. מכיוון שזה פרויקט שלך, אתה גם ה"בוס" שמאשר את הקוד:

5. **מיזוג (Merge):** בעמוד הראשי של ה-PR, לחץ על הכפתור הירוק **Merge pull request** ואז על **Confirm merge**.

השלמת המעגל במחשב שלך (קריטי להמשך): אחרי שמיזגת את הקוד ב-GitHub, השרת מעודכן אבל המחשב שלך עדיין לא יודע על זה. חזור לטרמינל ב-PyCharm והרץ את הפקודות הבאות כדי לסנכרן את ה-main המקומי שלך ולהיות מוכן לשבוע הבא:

```
git checkout main
git pull
```

ברגע שהפעולות האלו הסתיימו – שבוע 7 סגור וחתום רשמית, וה-main שלך מעודכן ונקי.

User prompt: יש בדיקות ? ואז איך להגיש ?

Response: לפי המסמך "Python task 02 (5).pdf - 06" שהעלית, יש הנחיות מפורטות מאוד לגבי תהליך ההגשה ומערכת הבדיקות. הנה מה שאתה צריך לעשות:

איך להגיש?

- **הוספת מרצה/בודק (קריטי):** עליך להוסיף את המשתמש talbarfm ל-Repository שלך בגיטהאב עם הרשאות Admin (מנהל), אחרת העבודה שלך לא תיבדק.
- **פתיחת Pull Request:** ההגשה עצמה מתבצעת על ידי פתיחת Pull Request (בדיוק כפי שהכנו את השטח לעשות).
- **פורמט הקבצים:** יש להגיש כל תרגיל כקובץ מודול נפרד (בפורמט .py בלבד, לא .ipynb).

- **שמות הקבצים והפונקציות:** שם הקובץ חייב להיות במבנה המדויק <week>.<part>.<exercise name>.py, ושם הפונקציה בתוך הקובץ חייב להיות תואם בדיוק לשם התרגיל.
- **בלוק הבדיקה:** חובה להשתמש ב-'__name__ == '__main__' בכל קובץ (מצוין שכבר יישמנו את זה בתרגיל האחרון!).
- **הגשת שיחות AI (חשוב!):** לפי נהלי השימוש ב-AI, אתה חייב להוריד את שיחות ה-AI שלך (כמו השיחה הזו) כקובץ PDF, ולהעלות אותן לתיקייה בשם ai_conversations/ בתוך ה-Repository שלך. חובה להגיש לפחות קובץ שיחה אחד.

האם יש בדיקות?

- **כן, יש בדיקות אוטומטיות:** המסמך מציין שיש בדיקות אוטומטיות של **Pylint** (שבודק את תקניות ונראות הקוד) ושל **Pytest** (שבודק שהקוד עובד נכון).
- **מתי הן רצות?** הבדיקות האלו מופעלות באופן אוטומטי על כל Pull Request שאתה פותח.
- **קוד נקי:** מעבר לנכונות הקוד, הציון שלך מורכב גם מעמידה בהנחיות לקוד נקי (Clean Code).

אז לפני שאתה מסיים סופית, תוודא שפתחת את תיקיית ה-ai_conversations/ והכנסת אליה את ה-PDF, ששמות הקבצים תקינים, ושהענקת הרשאות ניהול ל-talbarfm. ברגע שכל זה מוכן – אתה פותח את ה-Pull Request והבדיקות האוטומטיות יתחילו לרוץ!

User prompt: איך אפשר לראות את הבדיקות הללו? נגיד שאין לי פתאום טעות על שם לא נכון של קובץ ובסגנון

Response: הבדיקות האוטומטיות של Pylint ו-Pytest מופעלות ומוצגות ישירות בתוך ה-Pull Request בגיטהאב מיד אחרי שאתה פותח אותו.

כך תוכל לראות אותן ולוודא שאין לך שגיאות קטנות כמו שם קובץ שגוי:

- **בתחתית עמוד ה-Pull Request:** ברגע שתלחץ על "Create pull request", תראה אזור חדש שמופיע בתחתית העמוד שבו כתוב "Checks".
- **סטטוס הבדיקות:** בזמן שהשרת בודק את הקוד תראה עיגול צהוב מסתובב. כשהבדיקה תסתיים, תקבל או ו ירוק (הכל עבר בהצלחה) או איקס אדום (הבדיקה נכשלה).
- **כניסה לפרטים (Logs):** אם קיבלת איקס אדום, לחץ על הקישור **Details** שלידו. זה יפתח לך את מסך הטרמינל של השרת של גיטהאב, ושם תוכל לראות בדיוק מה קראס. למשל, אם המערכת חיפשה קובץ ולא מצאה אותו בגלל שגיאת כתיב בשם, אתה תראה שם שגיאה ברורה על כך שהקובץ חסר.
- **תיקון ודחיפה מחדש:** אם גילית טעות כזו, אתה פשוט חוזר ל-PyCharm, מתקן את השם, עושה שוב git add, git commit ו-git push. ה-Pull Request יתעדכן אוטומטית והבדיקות ירוצו מחדש.
- **מסמך העזר בפרויקט:** לפי קובץ ההנחיות, בתוך ה-Repository ששיכפלת קיים קובץ בשם GITHUB-CHECKS.md שמסביר בדיוק איך עובדות הבדיקות האוטומטיות האלו שרצות על כל Pull Request. שווה להציץ בו כדי להבין לעומק מה הן מחפשות.

טיפ לבדיקה מקומית (לפני ה-Push): אם אתה רוצה לוודא שהקוד שלך עומד בסטנדרטים של קוד נקי עוד לפני שאתה מעלה אותו לשרת, אתה יכול להריץ את הלינטר אצלך. בטרמינל של PyCharm (היכן שהרצת את ה-Git), הקלד:

```
pylint 7.2.sent_turtle.py
```

הפקודה הזו תסרוק את הקובץ שלך ותיתן לך חיווי מיידי (וציון מ-0 עד 10) על בעיות של רווחים, שמות משתנים, או

חוסר ב-Docstrings, בדיוק כפי שהבדיקה האוטומטית בגיטהאב תעשה.

User prompt: לא הבנתי איפה ?

Response: אתה נמצא בדיוק במסך הנכון!

תסתכל למעלה בצד ימין של המסך שהעלית, יש שם כפתור ירוק שכתוב עליו **"New pull request"**. עליך ללחוץ עליו כדי להתחיל.

ככה ממשיכים מכאן:

1. אחרי הלחיצה, גיטהאב יראה לך מסך השוואה. תוודא שמוגדר שם להעביר מהענף week7 (ב-compare) לענף main (ב-base).
2. יופיע כפתור ירוק נוסף של **"Create pull request"** – תלחץ עליו.
3. תן כותרת עניינית (למשל Week 7) ותלחץ שוב על **"Create pull request"** לאישור סופי.

רק **אחרי** שתסיים את התהליך הזה וה-Pull Request יפתח רשמית, העמוד ישתנה ואתה תראה בתחתית שלו את חלונית ה-**Checks** עם הבדיקות האוטומטיות שדיברנו עליהן קודם. כרגע הרשימה ריקה פשוט כי עוד לא פתחת את בקשת המיזוג.

User prompt: עשיתי new pull request לא מופיע כלום

Response: התמונה מסבירה בדיוק מה קרה. אתה כרגע מנסה להשוות בין שני דברים זהים, ובנוסף המערכת מנסה לפתוח את ה-Pull Request מול המאגר המקורי של הקורס במקום מול המאגר שלך.

כדי לסדר את זה ולראות את הקוד שלך, עליך לשנות את ההגדרות בתפריטים הנגללים (Dropdowns) שמופיעים בשורה האפורה למעלה:

1. **שינוי מאגר היעד (צד שמאל):** לחץ על התפריט שכתוב בו כרגע base repository: excellenteam-scaleup/python-t.... שנה אותו למאגר שלך: JonathanSchmidtt/python_ex_indi. ברגע שתעשה את זה, הדף יתרענן.
2. **שינוי הענף להשוואה (צד ימין):** לאחר הרענון, לחץ על התפריט שכתוב בו compare: main. שנה אותו לענף שעבדת עליו ודחפת עכשיו: week7.

ברגע שתבחר ב-week7, המסך ישתנה מיד. אתה תראה למטה את כל הקבצים ששינית, ויופיע כפתור ירוק של **Create pull request**. לחץ עליו, תן כותרת כמו Week 7, ואשר את הפתיחה.

User prompt: ככה?

Response: כמעט! פתרת את הבעיה של המאגר והגעת למקום הנכון, אבל תסתכל על שני התפריטים בשורה העליונה: מוגדר שם שאתה משווה את base: main מול compare: main. מכיוון שאתה משווה את אותו ענף לעצמו, גיטהאב אומר לך "There isn't anything to compare".

כדי לראות את השינויים שלך ולפתוח את הבקשה:

1. לחץ על הכפתור מצד ימין שכתוב עליו **compare: main**.
2. בחר מהרשימה את הענף **week7** (שים לב שהוא אפילו מוצע לך בנוחות למטה תחת "Example comparisons").

ברגע שתבחר ב-week7, המסך יתרענן, אתה תראה את כל שינויי הקוד שעשינו היום בתיבת הדואר, ויופיע הכפתור הירוק של **Create pull request**. לחץ עליו, תן כותרת מתאימה, ותוכל לראות את הבדיקות האוטומטיות מתחילות לרוץ.

User prompt: ככה?

Response: ממש קרוב! עכשיו בחרת את ענף week7 גם ביעד וגם במקור. בגלל שאתה משווה ענף לעצמו, גיטהאב אומר שוב שאין מה להשוות.

כדי לפתוח את ה-Pull Request כמו שצריך, השאר את ענף **week7** בצד ימין (תחת **compare**), אבל שנה את הענף בצד שמאל (תחת **base**) חזרה ל-**main**.

ברגע שתגדיר base: main מול compare: week7, יופיע לך כפתור "Create pull request" שתוכל ללחוץ עליו.

User prompt: אוקיי זה מה שיש עכשיו : ללחוץ? ומה לגבי ה WEEK5 WEEK6?

Response: בדיוק ככה! אתה יכול ללחוץ בלב שלם על הכפתור הירוק **"Create pull request"**. ההגדרה עכשיו מושלמת (base: main מול compare: week7), והמערכת מסמנת לך בירוק שאין התנגשויות (Able to merge).

לגבי השאלה המצוינת שלך על כך שאתה רואה שם את ה-Commits של שבועות 5 ו-6: זה קורה כי גיטהאב משווה את הענף week7 הנוכחי שלך לענף main, ורואה שכל התרגילים של שבועות 5 ו-6 עדיין לא קיימים ב-main המרכזי. מכיוון שבהוראות המטלה (Task 02) התבקשת להגיש את כל 11 התרגילים של שבועות 5, 6 ו-7 כמקשה אחת, זה בסדר גמור והגיוני לחלוטין שכולם נדחפים וממוזגים עכשיו בבת אחת ב-Pull Request הזה.

מה לעשות עכשיו?

1. לחץ על **Create pull request**.
2. תן ל-PR כותרת מסודרת שמסבירה מה יש בו, למשל: Task 02 - Weeks 5, 6, and 7.
3. לחץ שוב על אישור כדי לפתוח את הבקשה רשמית.
4. גלול לתחתית העמוד שייפתח כדי לראות את חלונית ה-Checks (הבדיקות האוטומטיות של Pylint ו-Pytest) מתחילות לרוץ על כל הקבצים.

User prompt: וכאן ללחוץ שוב ?

Response: כן, בדיוק! עליך ללחוץ על הכפתור הירוק **"Create pull request"** שמופיע למטה.

ברגע שתלחץ עליו, הבקשה תיווצר באופן סופי ואתה תועבר לעמוד של ה-Pull Request עצמו. בעמוד החדש שייפתח תוכל לגלול למטה, ושם סוף סוף תראה את הבדיקות האוטומטיות (ה-Checks של Pylint ו-Pytest)

מתחילות לרוץ על כל הקבצים שהעלית. בהצלחה!

User prompt: ללחוץ : זה לי הופיע ועכשיו?

Response: כן, הגעת בדיוק לשלב הסופי! גיטהאב זיהה בהצלחה את כל ה-Commits שלך (כולל האחרון של תיקון השמות ותיבת הדואר), והכפתור הירוק **"Merge pull request"** יחד עם ההודעה "No conflicts with base branch" אומרים שהכל תקין ומוכן למיזוג לענף הראשי.

אבל רגע אחד לפני שאתה לוחץ! בהוראות של הקורס (שקראנו קודם) יש דרישה קריטית להגשה: **להעלות את קובץ ה-PDF של שיחת ה-AI שלנו לתיקיית ai_conversations/**.

- **אם טרם העלית את קובץ ה-PDF לענף week7:** אל תלחץ עדיין! חזור ל-PyCharm, צור את התיקייה, שמור בתוכה את השיחה שלנו כ-PDF, ותעשה שוב את התהליך הרגיל (git add ., ואז git commit -m "Add AI conversation", ובסוף git push). ה-Pull Request הזה פתוח ויתעדכן אוטומטית ברגע שתדחוף את הקובץ.
- **אם כבר הוספת את ה-PDF מראש / אם אינך נדרש להוסיף אותו לתרגיל הזה:** מצוין! אתה יכול ללחוץ בלב שלם על **"Merge pull request"**. לאחר שתלחץ עליו, הוא יתחלף לכפתור **"Confirm merge"** – לחץ גם עליו כדי לאשר סופית את הכנסת הקוד ל-main.

(הערה קטנה: בתמונה שהעלית לא רואים אזור של חלונית ה-Checks שדיברנו עליה. אם המרצים הגדירו בדיקות אוטומטיות, ייתכן שהן נמצאות נמוך יותר בעמוד או שהן יופעלו רק אחרי המיזוג. מבחינת הפעולה שלך בגיטהאב – הדרך פנויה).

ברגע שתסיים לאשר את המיזוג באתר, אל תשכח לחזור לטרמינל ב-PyCharm, לעבור לענף הראשי ולמשוך את העדכונים כדי שהמחשב שלך יסתנכרן עם השרת וידע ששבוע 7 מוזג בהצלחה:

```
git checkout main
git pull
```
